



Trabajo Práctico N^o2: Filtrado digital de señales

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Julio de 2025

Resumen

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
1.2. Aspectos a tener en cuenta	3
2. Marco teórico	3
3. Placa de Desarrollo y Herramientas Utilizadas	3
3.1. Microcontrolador STM32F446RE	3
3.2. Periféricos clave utilizados	4
3.2.1. ADC (Analog-to-Digital Converter)	4
3.2.2. DAC (Digital-to-Analog Converter)	4
3.2.3. DMA (Direct Memory Access)	5
3.2.4. TIM (Timers)	5
3.2.5. GPIO – General Purpose Input/Output	5
3.2.6. Herramientas de desarrollo	5
4. Diseño del programa	7
5. Programa principal	7
5.1. Configuración de periféricos	7
5.1.1. Configuración del ADC	7
5.1.2. Configuración del DAC	8
5.1.3. Configuración del temporizador	8
5.1.4. Configuración del DMA	8
5.1.5. Configuración de GPIO	9
5.1.6. Inicialización de los filtros digitales	9
5.1.7. Ejecución en tiempo real	9
6. Resultados experimentales	9
7. Conclusiones	9

1. Introducción

1.1. Objetivos del proyecto

1.2. Aspectos a tener en cuenta

2. Marco teórico

3. Placa de Desarrollo y Herramientas Utilizadas

El proyecto fue desarrollado sobre la placa STM32 Nucleo-F446RE, basada en el microcontrolador STM32F446RE, perteneciente a la familia STM32F4 de STMicroelectronics. Esta plataforma integra una arquitectura avanzada ARM Cortex-M4, periféricos orientados a procesamiento de señales, y herramientas de desarrollo que permiten implementar sistemas en tiempo real de forma eficiente.

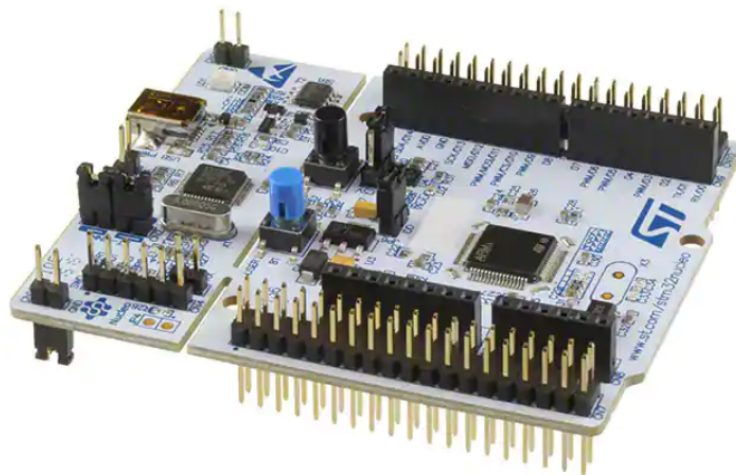


Figura 1: Placa de desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de 32 bits basado en el núcleo ARM Cortex-M4, optimizado para procesamiento digital con soporte de instrucciones DSP y unidad de punto flotante opcional (FPU simple precisión). Sus principales características relevantes al proyecto son:

- Frecuencia de operación: hasta 180MHz
- Memoria Flash: 512KB

- RAM: 128KB
- Tensión de operación: 1.8V a 3.6V
- Paquete: LQFP64 (compatible con la Nucleo-F446RE)

3.2. Periféricos clave utilizados

3.2.1. ADC (Analog-to-Digital Converter)

El STM32F446RE cuenta con 3 ADCs independientes (ADC1, ADC2 y ADC3), cada uno de 12 bits de resolución y con capacidad de hasta 2.4 MSPS (mega muestras por segundo) en modo intercalado. Para este proyecto se utiliza el ADC1 en modo regular y con DMA activado. Sus características técnicas relevantes incluyen:

- Resolución seleccionable: 6, 8, 10 o 12 bits (se usa 12 bits para mayor precisión).
- Tasa de muestreo máxima (single conversion): hasta 2.4 MSPS.
- Rango de entrada: 0V a VREF (típicamente 3.3V).
- Tiempo de muestreo configurable: desde 3 hasta 480 ciclos de ADC.
- Trigger externo: el inicio de la conversión puede ser disparado por un temporizador, lo cual permite muestrear a frecuencias precisas.
- Modo DMA: transferencia automática de los resultados a un buffer en memoria (sin intervención del CPU).
- Modo continuo + DMA circular: permite muestreo ininterrumpido con mínimo overhead.

3.2.2. DAC (Digital-to-Analog Converter)

El STM32F446RE incluye 2 canales DAC (DAC1 y DAC2), ambos de 12 bits, con capacidad de salida en modo bufferizado (con baja impedancia) o sin buffer (alta velocidad). Se utiliza DAC1 en este proyecto para convertir las muestras digitales de salida nuevamente en una señal analógica. Características clave:

- Resolución de conversión: 12, 10 u 8 bits.
- Velocidad típica: hasta 1 MSPS (modo sin buffer).
- Modo bufferizado: reduce el ruido, ideal para salidas de audio.
- Trigger externo: puede sincronizarse con un temporizador (por ejemplo TIM6/TRGO).
- Soporte DMA: se permite la alimentación directa de datos desde memoria al DAC sin CPU.
- Salidas disponibles: a través de pines analógicos dedicados (por ejemplo, PA4 para DAC1).

3.2.3. DMA (Direct Memory Access)

El controlador DMA2 es utilizado para las transferencias entre ADC/DAC y RAM. El STM32F446RE cuenta con dos controladores DMA (DMA1 y DMA2), con un total de 16 canales (streams) y soporte para múltiples periféricos. Características específicas:

- Transferencia en background, sin intervención del CPU.
- Modo circular: ideal para buffers cíclicos o doble buffering.
- Prioridad programable por canal.
- Transferencia por palabras (16 bits en Q15).
- Eventos por mitad de buffer y buffer completo: permite usar doble buffer sincronizado.

3.2.4. TIM (Timers)

El STM32F446RE cuenta con 14 temporizadores (4 avanzados, 2 básicos, 8 generales), todos con configuraciones de prescaler, auto-reload y posibilidad de generar eventos de actualización. Se utilizan para generar señales periódicas que actúan como trigger de muestreo. Características clave:

- TIM2-TIM5: de 32 bits, con amplio rango de conteo (útiles para precisión en frecuencias bajas).
- Configuración de frecuencia: vía Prescaler y ARR (Auto-Reload Register).
- Trigger de ADC/DAC: mediante señal TRGO (Trigger Output).
- Modo master-slave: permite sincronización entre periféricos.

3.2.5. GPIO – General Purpose Input/Output

Los pines GPIO permiten la interacción con el usuario mediante botones (teclas) y LEDs. Se utilizan:

- Velocidad de salida seleccionable: Low, Medium, Fast, High.
- Pull-up/Pull-down configurables.
- Interrupciones por flanco ascendente, descendente o ambos.

3.2.6. Herramientas de desarrollo

Para la implementación del proyecto se utilizaron las siguientes herramientas:

- STM32CubeIDE: entorno de desarrollo oficial de STMicroelectronics, basado en Eclipse, que permite configurar periféricos gráficamente, generar código y compilar con GCC.
- STM32CubeMX: herramienta para la configuración inicial de pines, relojes, periféricos y generación de código base.

- CMSIS y CMSIS-DSP: bibliotecas proporcionadas por ARM para facilitar el uso de instrucciones de bajo nivel y rutinas DSP optimizadas en punto fijo.
- GDB y ST-Link: utilizados para depuración, profiling y medición de tiempos mediante DWT->CYCCNT.

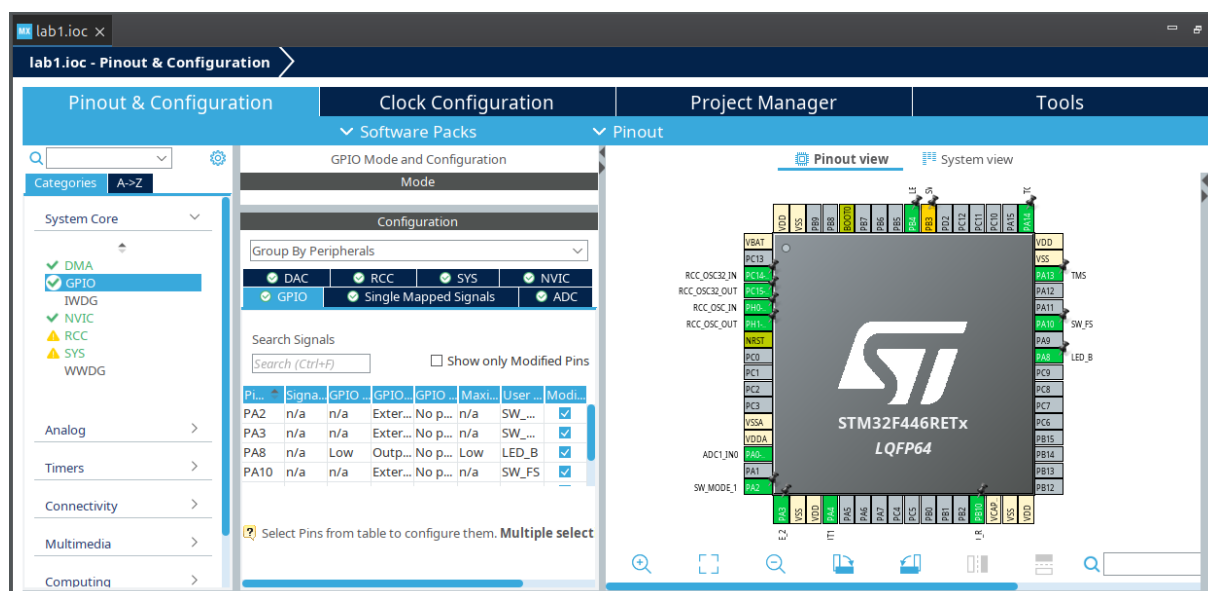


Figura 2: Entorno de desarrollo STM32CUBE IDE

4. Diseño del programa

5. Programa principal

5.1. Configuración de periféricos

El programa principal inicializa los periféricos necesarios para realizar la adquisición, procesamiento y reproducción de la señal en tiempo real. La arquitectura general mantiene el mismo esquema del Trabajo Práctico N^o1, incorporando ahora la etapa de filtrado digital mediante la biblioteca CMSIS-DSP.

La inicialización de los periféricos se realiza mediante las funciones generadas automáticamente por STM32CubeMX:

- MX_GPIO_Init()
- MX_DMA_Init()
- MX_ADC1_Init()
- MX_TIM2_Init()
- MX_DAC_Init()

Cada uno de estos periféricos cumple una función específica dentro del sistema de procesamiento digital en tiempo real.

5.1.1. Configuración del ADC

El conversor analógico-digital (ADC1) se utiliza para adquirir la señal de entrada con una resolución de 12 bits. El ADC es disparado periódicamente por el temporizador TIM2, lo que garantiza una frecuencia de muestreo precisa.

La transferencia de datos se realiza mediante DMA en modo circular, permitiendo almacenar continuamente las muestras en un buffer sin intervención directa del CPU.

La inicialización del ADC en modo DMA se realiza con:

```
1  HAL_ADC_Start_DMA(&hadc1,  
2      (uint32_t*) adc_buffer,  
3      BUFFER_SIZE);
```

El uso de DMA permite implementar un esquema de doble buffer, donde la mitad del buffer puede procesarse mientras la otra mitad continúa llenándose con nuevas muestras.

5.1.2. Configuración del DAC

El convertor digital-analógico (DAC1) se utiliza para reconstruir la señal procesada y permitir su visualización mediante un osciloscopio.

El DAC también opera en modo DMA, lo que permite enviar continuamente los datos procesados desde memoria hacia la salida analógica sin intervención del CPU.

La inicialización del DAC se realiza mediante:

```
1  HAL_DAC_Start_DMA(&hdac,  
2      DAC_CHANNEL_1,  
3      (uint32_t*) dac_buffer,  
4      BUFFER_SIZE,  
5      DAC_ALIGN_12B_R);
```

5.1.3. Configuración del temporizador

El temporizador TIM2 se utiliza como base de tiempo para definir la frecuencia de muestreo del sistema. Este temporizador genera eventos periódicos (TRGO) que disparan simultáneamente el ADC y el DAC, garantizando sincronización entre adquisición y reconstrucción de la señal.

La frecuencia de muestreo puede modificarse dinámicamente mediante interrupciones externas, ajustando el valor del registro Auto-Reload del temporizador.

El temporizador se inicia mediante:

```
1  HAL_TIM_Base_Start(&htim2);
```

Los valores de autoreload utilizados para cada frecuencia de muestreo son:

- 8 kHz
- 16 kHz
- 22 kHz
- 44 kHz
- 48 kHz

5.1.4. Configuración del DMA

El controlador DMA permite transferir datos entre los periféricos y la memoria sin intervención del CPU. En este sistema se utiliza para:

- Transferir muestras desde el ADC hacia el buffer de entrada
- Transferir muestras procesadas hacia el DAC

El uso de DMA reduce la carga del procesador y permite garantizar procesamiento en tiempo real.

5.1.5. Configuración de GPIO

Los pines GPIO se utilizan para interactuar con el usuario mediante pulsadores y LEDs.

Las interrupciones externas permiten:

- Cambiar la frecuencia de muestreo
- Cambiar el tipo de filtro (LP, HP, BP, BS)
- Activar el modo bypass del filtro

Además, los LEDs RGB indican visualmente el estado del sistema y la frecuencia de muestreo seleccionada.

5.1.6. Inicialización de los filtros digitales

Luego de inicializar los periféricos, se configuran las estructuras necesarias para implementar los filtros digitales en formato Q15 utilizando la biblioteca CMSIS-DSP.

La inicialización de los filtros se realiza mediante:

```
1  arm_fir_init_q15(&FPB,  
2      FPB_8K_NUM_TAPS ,  
3      FPB_8K_COEFFS ,  
4      FPB_status ,  
5      BUFFER_SIZE / 2);
```

Se definen filtros paso bajo, paso alto, pasa banda y rechazo de banda para cada frecuencia de muestreo soportada por el sistema.

5.1.7. Ejecución en tiempo real

Una vez inicializados los periféricos, el sistema comienza a operar en tiempo real mediante interrupciones del DMA. El procesamiento de la señal se realiza dentro de las funciones callback:

- HAL_ADC_ConvHalfCpltCallback()
- HAL_ADC_ConvCpltCallback()

Estas funciones procesan cada mitad del buffer utilizando los filtros digitales seleccionados.

6. Resultados experimentales

7. Conclusiones